A Guided Tour of the Theory for the Problem-Solving Model

Three contributions, every step explained

Working note — prepared for Fausto

June 16, 2026

Abstract

This note collects, in one place and in plain language, the three rounds of analytical work on our agent-based model of distributed problem solving: Atiyab's first "self-averaging" mean-field, Atiyab's second note (a dynamical rate equation plus an *exact* pair-level framework), and Juan's recent "beyond mean-field" calculation of the steady state. It is written for a reader who is comfortable with the *model* but not with theoretical-physics-style derivations. Every symbol is defined, every step is motivated, and after each contribution I say plainly *what it gets right, what it gets wrong, and why.* The aim is that by the end you can see exactly how the three pieces fit together and where the open frontier is.

Contents

1 What we are trying to predict, and what "theory" means here

Our model is a simulation: N agents on a fixed directed network, each holding a binary vector $x^{(i)} \in \{0,1\}^K$ and a personal knowledge base of clauses, repeatedly learning clauses (by observation or from neighbours) and running a greedy repair after each one. We measure things like the average number of violated clauses. A *simulation* tells us *what* happens; a *theory* tries to tell us *why*, and ideally gives a formula that predicts the measured numbers without running the code.

The three quantities the theory tries to predict are:

- V — the **violation count**: how many of the $M = \alpha K$ universal clauses an agent's current assignment violates (lower is better).
- p — the **fraction of ones**: across all the bits in the system, what fraction are set to 1.
- H — the **homogeneity**: how much agents agree with each other, variable by variable ($H = 1$ is unanimous agreement).

The two clause types (used constantly below). Every clause involves two variables.

- An AND clause is *satisfied* only when *both* bits are 1; otherwise it is violated.
- An XOR clause is *satisfied* only when *exactly one* bit is 1 (the two bits differ); if the two bits are equal (00 or 11) it is violated.

Half the clauses are AND and half XOR.

In plain terms. Everything that follows is built on one trick: instead of tracking every one of the $N \times K$ individual bits, we summarise the whole system by a few average numbers (like p) and try to write down equations those averages must obey. That trick is called a *mean-field approximation*. The three contributions are increasingly careful versions of exactly this trick.

2 Atiyab's first contribution: the simple "self-averaging" mean-field

Source: `Self Average Approximation.pdf`.

2.1 The central hypothesis

After the simulation has run for a long time, each agent has seen and exchanged so many clauses that its knowledge base looks like a *random sample of the global clause pool*. And since all agents are sampling from the same pool, the bits they hold should settle into the same statistics everywhere. So we make a bold simplification:

Every bit in the system is 1 *with the same probability* p*, independently of every other bit.*

In plain terms. This is the "self-averaging" assumption. It throws away *who* is connected to *whom*, *which* agent holds *which* clause, and any correlation between bits. The entire state of the system is compressed into a single number p. It is obviously too crude to be exactly true — but the question is whether it is *useful*.

2.2 Step 1 — violation probability of one clause

If a bit is 1 with probability p, then for a single clause on two (independent) bits:

$$\Pr(\text{AND violated}) = 1 - \Pr(\text{both bits } 1) = 1 - p^2, \tag{1}$$

$$\Pr(\text{XOR violated}) = \Pr(\text{both equal}) = \underbrace{p^2}_{11} + \underbrace{(1-p)^2}_{00} = 1 - 2p(1-p). \tag{2}$$

2.3 Step 2 — expected total violations

There are $M = \alpha K$ clauses, half of each type, so the expected violation count is just $M/2$ times each probability, added up:

$$V(p) = \frac{M}{2}\big(1 - p^2\big) + \frac{M}{2}\big(p^2 + (1-p)^2\big) = M\left(1 - p + \frac{p^2}{2}\right). \tag{3}$$

2.4 Step 3 — find the best p

We now ask: which value of p makes V as small as possible? The standard tool is the *derivative* $\frac{dV}{dp}$, which measures the slope of V as we change p. At the bottom of a valley the slope is zero, so we set it to zero:

$$\frac{dV}{dp} = M\,(p-1) = 0 \quad \Longrightarrow \quad p^\star = 1. \tag{4}$$

To check this is a *minimum* (a valley) and not a maximum (a hill), we look at the second derivative $\frac{d^2V}{dp^2} = M > 0$, which is positive — confirming a valley. Plugging $p^\star = 1$ back in:

$$\boxed{V^\star = \frac{M}{2} = \frac{\alpha K}{2}}. \tag{5}$$

In plain terms. The prediction is striking: the long-run violation level should be $\alpha K/2$, depending *only* on the clause density α and the number of variables K — and *not* on the network, the number of agents N, the observation probability, or how often clauses are swapped.

2.5 What it got right, and what it got wrong

Right. The simulation's long-run violations track $\alpha K/2$ to within roughly 5–10% as α and K are varied, across both random and scale-free networks (the straight-line plots in Atiyab's note). And it correctly predicted *network independence* of the long-run level — which later became the central empirical finding of the paper.

Wrong (and this matters). The theory says $p^\star = 1$: every agent should drive *every* variable to 1. But:

- The simulation settles at $p \approx 0.7$, not 1.
- Homogeneity settles around $H \approx 0.74$, not 1.
- The formula *fails* when we vary the observation probability or the clause interval — precisely the regimes where the "independent bits" assumption breaks.

In plain terms. Why can the *level* $\alpha K/2$ be roughly right while the *location* $p = 1$ is clearly wrong? Because the curve (3) is shallow near its bottom. At the true $p \approx 0.7$ we get $V \approx 0.545\,M$, versus the predicted minimum $0.5\,M$ — only about 9% apart. So the simple theory lands in the right ballpark for V for the "lucky" reason that being wrong about p costs little in V. That is exactly the kind of coincidence a better theory has to explain rather than rely on.

2.6 Atiyab's own explanation of network independence

The note already gives the right intuition for *why topology drops out of the long run*:

1. `comm_scale` normalises incoming communication by the average in-degree, so every agent receives roughly the same *amount* of information per tick regardless of topology — topology changes *who* transmits, not the average *rate*.
2. Once everyone's knowledge base is a moving sample of the same global pool, the greedy step sees the same clause statistics everywhere, so the long-run target is identical on every node. The network can only *speed up or slow down* convergence.

That last sentence — *network affects speed, not destination* — is the seed of everything that follows, and of the paper's transient-vs-steady-state story.

3 Atiyab's second contribution: dynamics, and an exact pair-level framework

Source: `Mean Field Model_version1.pdf`. This note has two distinct halves: first it makes the simple theory *dynamical* (how fast, not just where); then it builds an *exact* replacement for the crude independence assumption.

3.1 Half A — the same attractor, seen as a flow

Instead of minimising V directly, Atiyab asks what the greedy step *does to a single bit*, on average. Consider testing a flip of variable j from $0 \to 1$. A neighbouring clause that contains j changes its violation status depending on the partner bit:

Clause type	partner = 1 (prob. p)	partner = 0 (prob. $1-p$)
AND, $0 \to 1$	$01 \to 11$: violated→satisfied, $\Delta = -1$	$00 \to 10$: stays violated, $\Delta = 0$
XOR, $0 \to 1$	$01 \to 11$: satisfied→violated, $\Delta = +1$	$00 \to 10$: violated→satisfied, $\Delta = -1$

Averaging over the partner bit, the *expected* change per clause is

$$\mathbb{E}_{\text{AND}}[\Delta(0 \to 1)] = -p, \qquad \mathbb{E}_{\text{XOR}}[\Delta(0 \to 1)] = 2p - 1. \tag{6}$$

Each variable sits in about α AND clauses and α XOR clauses, so the total expected change from flipping a zero up is

$$\Delta_{0\to1} \;\approx\; \alpha(-p) + \alpha(2p-1) = -\alpha(1-p) \;\leq 0. \tag{7}$$

A negative change means fewer violations, so a $0 \to 1$ flip is *always accepted* (until $p = 1$). Repeating the same bookkeeping for $1 \to 0$ gives $\Delta_{1\to0} \approx +\alpha(1-p) \geq 0$, so $1 \to 0$ flips are *never* accepted. The system therefore drifts towards $p = 1$ — the same attractor as before, now derived from the *dynamics* rather than from minimising a curve.

3.2 Half A — a rate equation and a convergence timescale

In plain terms. A *rate equation* is just "rate of change = (how often something happens) × (how much it changes things each time)." Writing $\frac{dp}{dt}$ for the rate of change of p in time, we can turn the drift above into a prediction of *how fast* the system converges.

Only $0 \to 1$ flips change p, and they are available in proportion to the fraction of zeros, $1 - p$. If an agent learns new clauses at rate r, then

$$\frac{dp}{dt} = \frac{2\gamma}{K}\, r\,(1-p), \tag{8}$$

where γ is a proportionality constant and the $2/K$ accounts for a clause touching 2 of the K variables. This is a textbook equation whose solution is an *exponential approach*:

$$p(t) = 1 - (1-p_0)\, e^{-(2\gamma r/K)\, t}, \qquad \text{timescale } \tau = \frac{K}{2\gamma r}. \tag{9}$$

So convergence is slower for more variables (K) and faster the more often an agent learns (r). Crucially, Atiyab writes

$$r = r_{\text{obs}} + r_{\text{int}}, \qquad r_{\text{int}} \propto \text{average in-degree}, \tag{10}$$

i.e. the learning rate splits into an observation part (an external parameter) and an interaction part (set by the network). **This is the first place the network enters a formula** — and it enters the *timescale*, exactly as the speed-not-destination intuition predicted. The same equation gives homogeneity $H(t) \approx \max(p, 1-p)$ rising smoothly to its attractor with the same timescale, explaining why the measured H curves are smoother than the V curves.

3.3 Half A — the honest admission

The note then states plainly: $p = 1$ *is not the real attractor* (the simulation converges to different values below 1 for different networks), so $H = \max(p, 1-p)$ cannot be the whole story either. Hence: "we need a better mean-field model." This is the hand-off to Half B.

3.4 Half B — the exact pair-level framework

The crude step was treating two bits in a clause as *independent*, e.g. writing $\Pr(\text{both } 1) = p^2$. But bits that co-occur in clauses are *correlated* (the greedy step couples them), so p^2 is wrong. The fix: stop factorising, and track the joint distribution of *pairs* of bits directly.

For a clause on variables (i, j) define

$$q_{ab}^{(ij)}(t) = \Pr\big(x_i = a,\, x_j = b\big), \qquad a, b \in \{0, 1\}. \tag{11}$$

In plain terms. $q_{11}^{(ij)}$ is the genuine probability that *both* bits of that clause are 1 — measured, not assumed to be p^2. Keeping these pair-probabilities instead of just p is the "pair mean-field."

In these terms the violation probabilities are *exact*:

$$v_{\text{AND}}^{(ij)} = 1 - q_{11}^{(ij)}, \qquad v_{\text{XOR}}^{(ij)} = q_{00}^{(ij)} + q_{11}^{(ij)} \;(\equiv q_{\text{eq}}^{(ij)}), \tag{12}$$

and averaging over all AND clauses (M_A of them) and all XOR clauses (M_X) gives an **exact identity** for the mean violation count:

$$V(t) = M_A\big(1 - q_{11}^{A}(t)\big) + M_X\, q_{\text{eq}}^{X}(t). \tag{13}$$

In plain terms. "Exact identity" means: *no approximation has been made.* If you knew the pair quantities q_{11}^A and q_{eq}^X, equation (13) would give the true V on the nose. The same is done for homogeneity, giving $H = \frac{1}{2} + \frac{1}{K}\sum_j |f_j - \frac{1}{2}|$, where f_j is the fraction of agents with bit $j = 1$; the old $H = \max(p, 1-p)$ is recovered only when all the f_j are tightly bunched together.

3.5 Half B — exact flip rules, verified in code

Atiyab then writes the violation change of a proposed flip *exactly*, by counting the clauses around variable j:

- $A_1(j)$ = AND clauses on j whose partner bit is 1 (similarly A_0, X_1, X_0).

A short, clean derivation gives the exact acceptance rules:

$$\text{flip } 0 \to 1 \text{ accepted} \iff A_1 + X_0 > X_1, \tag{14}$$

$$\text{flip } 1 \to 0 \text{ accepted} \iff X_1 > A_1 + X_0. \tag{15}$$

He checked these against a real simulation run — every accepted flip obeys the inequality, a 100% match. (This also exposes that the *first* model was wrong about the dynamics: it had only $0 \to 1$ flips ever being accepted, whereas the real model accepts some $1 \to 0$ flips too.)

3.6 Half B — where it stops, and why

We can now write the exact rate of change of V in terms of the pair quantities (differentiating (13)):

$$\frac{dV}{dt} = -M_A \frac{dq_{11}^{A}}{dt} + M_X \frac{dq_{\text{eq}}^{X}}{dt}. \tag{16}$$

But here is the wall: to know how the *pairs* $q^{(ij)}$ change, you need to know about *triples* of bits (because flipping j disturbs every pair attached to j); to get triples you need quadruples; and so on.

In plain terms. This is called the system being *"not closed."* The equation for the thing you want depends on a more complicated thing you don't have. It is the single most common obstacle in this style of theory. The pair framework is therefore *correct but not yet solvable* — it is an honest, exact scaffold with a gap in the middle.

Reassuringly, if you *do* force the factorisation $q_{11}^A \approx p^2$, $q_{\text{eq}}^X \approx p^2+(1-p)^2$, equation (13) collapses back to the simple $V = M(1-p+p^2/2)$ of §2. So the pair theory is a strict *generalisation* of the first one, not a competing model.

4 Juan's contribution: computing the steady state "beyond mean field"

Source: `clauses.pdf` (email "Notes beyond mean field", 16 June 2026).

Juan attacks the precise thing the first theory got wrong — the claim $p^\star = 1$ — by *not assuming* where the system settles and instead *computing* it from the acceptance behaviour of the greedy step.

4.1 Step 1 — a rate equation built from pair-flips

When a new clause lands on a variable pair, that pair is in state 00, 10, or 11 with probabilities (under the independence assumption, as a first pass)

$$\Pr(00) = (1-p)^2, \quad \Pr(10) = 2p(1-p), \quad \Pr(11) = p^2. \tag{17}$$

Let F_{00} be the probability that the greedy step *accepts* flipping a 00 pair up to 11 (which adds two ones), and F_{11} the probability of accepting a $11 \to 00$ flip (which removes two ones); a 10 flip doesn't change the count. The rate of change of p is then

$$\boxed{\frac{dp}{dt} = 2(1-p)^2 F_{00} - 2p^2 F_{11}} \qquad \text{(Juan's Eq. 1).} \tag{18}$$

In plain terms. This is the *same kind* of rate equation as Atiyab's (8), but honest: instead of assuming every beneficial flip happens, it carries the actual *acceptance probabilities* F_{00} and F_{11}. Those are where the real physics hides.

4.2 Step 2 — the steady state

In plain terms. "Steady state" means the averages have stopped changing: $\frac{dp}{dt} = 0$. It does *not* mean individual agents stop flipping bits — they keep churning — only that the *population average* p holds still, like a river that flows while its water level stays constant.

Setting (18) to zero and solving for the steady value $\bar{p}$:

$$0 = 2(1-\bar{p})^2 F_{00} - 2\bar{p}^2 F_{11} \implies \frac{\bar{p}^2}{(1-\bar{p})^2} = \frac{F_{00}}{F_{11}} \implies \bar{p} = \frac{\sqrt{F_{00}}}{\sqrt{F_{00}} + \sqrt{F_{11}}}. \tag{19}$$

So the resting fraction of ones is decided by the *ratio* of the two acceptance rates. If flips up and flips down were equally easy, $\bar{p}$ would be $1/2$; the asymmetry between F_{00} and F_{11} pushes it higher.

4.3 Step 3 — violations from $\bar{p}$

Using the same per-clause probabilities as everyone else,

$$V = \tfrac{1}{2}\big(1-\bar{p}^2\big) + \tfrac{1}{2}\big(\bar{p}^2 + (1-\bar{p})^2\big) \qquad \text{(Juan's Eq. 2),} \tag{20}$$

as a *fraction* per clause. (Check: at $\bar{p} = 0.66$ this gives $V \approx 0.558$ — matching his quoted "$V \approx 0.55$".)

4.4 Step 4 — actually computing the acceptance rates F_{00}, F_{11}

This is the heart of Juan's note. To decide whether flipping a 00 pair up to 11 is accepted, he counts the net change in violations it causes among the *neighbouring* clauses, then asks "is the net change negative?"

Each variable sits in α AND and α XOR neighbouring clauses. Tracking which become satisfied or violated when a zero flips up (the same bookkeeping as Atiyab's Half A, but kept as counts) gives, for the two flipped zeros together, an average of

$$\mathbb{E}[N^+] = 2\alpha p \text{ (new violations created)}, \qquad \mathbb{E}[N^-] = 2\alpha \text{ (violations removed)}. \tag{21}$$

In plain terms. Poisson & Skellam, briefly. The *number* of created/removed violations is random, not exactly its average. A *Poisson distribution* is the standard model for "a count of independent rare-ish events with a known average." The *difference* of two Poisson counts (here, created minus removed) is called a *Skellam distribution*. Juan uses it to get the probability distribution of the *net* violation change.

So the net change for a 00 flip is

$$Z_{00} = \text{Poisson}(2\alpha p) - \text{Poisson}(2\alpha), \tag{22}$$

and he writes $H_{00}(k) = \Pr(Z_{00} \le k)$ for the probability that the neighbour change is at most k. The flip is accepted when the *total* change (neighbours *plus* the new clause itself *plus* the clause displaced by the knowledge-base eviction) is negative. Folding in whether the incoming clause is AND or XOR, and whether the displaced clause was satisfied, yields

$$F_{00} = \tfrac{1}{2}\Big[(1-\bar{p}^2)H_{00}(1) + \bar{p}^2 H_{00}(0)\Big] + \tfrac{1}{2}\Big[(\bar{p}^2 + (1-\bar{p})^2)H_{00}(0) + (1-\bar{p}^2-(1-\bar{p})^2)H_{00}(-1)\Big], \tag{23}$$

and a mirror-image expression for F_{11}.

In plain terms. The only subtle modelling choice here is the "displaced clause" bookkeeping — because each learned clause also evicts an old one (the FIFO cap in the code), each acceptance branch is shifted by ± 1 in the threshold $H_{00}(\cdot)$. This is the part most worth double-checking against the simulation; the rest is clean counting.

4.5 Step 5 — solve and compare

Substituting F_{00}/F_{11} into (19) gives one equation with $\bar{p}$ on both sides (an *implicit equation*); solving it numerically gives $\bar{p}$, hence V, as a function of α. The results:

- at $\alpha = 4$: $\bar{p} \approx 0.66$, $V \approx 0.55$ — in line with the simulation;
- as $\alpha \to \infty$: $\bar{p} \to 1$, recovering the old mean-field value.

In plain terms. This is the breakthrough. The simple theory *assumed* $p = 1$ and was wrong; Juan *derives* $p \approx 0.66$, which is exactly the ~ 0.7 the simulation has always shown (and which our per-variable heat-map already hinted at). And the old theory reappears naturally as the dense-clause ($\alpha \to \infty$) limit — so nothing is contradicted, the picture is just completed.

5 How the three pieces fit together

Contribution	What it delivers	Its limitation
Atiyab #1 (simple MF)	The *level* of violations, $V^\star = \alpha K/2$, and the prediction (later confirmed) that the long run is network-independent.	Predicts $p = 1$, which is false; only "accidentally" close on V; fails vs. obs. prob. and clause interval.
Atiyab #2 (dynamics + pairs)	(A) A *timescale* $\tau = K/2\gamma r$ with $r = r_{\text{obs}} + r_{\text{int}}$ and $r_{\text{int}} \propto$ in-degree — the network's first appearance in a formula. (B) An *exact* pair-level identity for V and exact, code-verified flip rules.	The dynamic part still rides on the wrong $p = 1$ attractor. The exact part is *not closed*: pair equations need triples, which are not derived.
Juan (beyond MF)	The correct *steady-state* location: $\bar{p} = \sqrt{F_{00}}/(\sqrt{F_{00}} + \sqrt{F_{11}})$ with F_{00}, F_{11} computed from a Poisson/Skellam model. Matches sim ($\bar{p} \approx 0.66$ at $\alpha = 4$); recovers old MF as $\alpha \to \infty$.	Steady-state only. Still uses the independent-pair assumption for Pr(00) etc. as a first pass; and the dynamic/network-dependent version is not yet done (his explicit open question).

5.1 The open frontier (where this is heading)

Juan's email asks the right next question: *how do we make this dynamic and network-dependent?* His own suggestion — let the learning rate depend on a node's degree and make p time-dependent — joins directly onto Atiyab's $r = r_{\text{obs}} + r_{\text{int}}$ with $r_{\text{int}} \propto$ in-degree. Putting Juan's honest steady-state machinery into Atiyab's rate-equation form gives, per agent,

$$\frac{dp_i}{dt} = r_i\, G(p_i), \qquad G(p) = 2(1-p)^2 F_{00}(p) - 2p^2 F_{11}(p), \tag{24}$$

where r_i is agent i's learning rate (degree-dependent). Two observations fall straight out:

1. **The steady state $\bar{p}$ (where $G = 0$) does not depend on r_i.** The rate multiplies the whole right-hand side, so it cancels when we set it to zero. Every agent — hub or leaf, in any topology — relaxes to the *same* $\bar{p}$. *This is an analytical proof of the paper's headline result* (long-run performance is topology-independent), which the draft currently only asserts.
2. **The rate r_i controls the *speed*, not the destination.** The convergence time is $\tau_i \propto 1/r_i$, so all the network/transient differences live here — exactly Atiyab's "network affects speed not destination," now on a correct steady state.

There is a clean, testable refinement to chase: because `comm_scale` fixes the *average* rate equal across topologies, the transient differences the paper measures must come from the *spread* of r_i and from how fast clause *content* diffuses across the graph — i.e. from the network's mixing time / spectral gap. That predicts $t_{\text{conv}} \propto 1/(\text{spectral gap})$ with a gap-independent steady state, which we can check directly against the topology results we already have.

Suggested immediate step. Re-implement Juan's implicit equation for $\bar{p}(\alpha)$ in Python, reproduce his p and V curves, and overlay the actual simulation at $\alpha = 2$ (the paper's value) and $\alpha = 4$ (his value). That both verifies the new theory and turns it into a figure for the paper.